Informe de laboratorio 05

Tema: Programación para el microprocesador INTEL - Instrucciones Aritméticas

Nota

Estudiante	Escuela	Asignatura
Camilo B.A. Capatinta Almanza (Teoría grupo B) ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Lab. Arq. de computadoras Semestre: IV Código: 1702227

Laboratorio	Tema	Duración
05	Programación para el microprocesador INTEL - Instrucciones Aritméticas	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	20/10/2026	–/10/2026

Índice

1. Objetivos **3**

2. Materiales y equipos **3**

3. Actividades **3**
3.1. Preparación del entorno . . . 3
3.2. Primer programa: suma con ADD y ADC . . . 4
3.2.1. Acarreo invertido con `ADD` . . . 6
3.2.2. Variante con `ADC` . . . 6
3.2.3. Datos modificados . . . 7
3.3. Segundo programa: resta con SUB y SBB . . . 8
3.3.1. Acarreo activado manualmente . . . 9
3.4. Tercer programa: multiplicación con MUL e IMUL . . . 12
3.5. Cuarto programa: división con DIV e IDIV . . . 14
3.5.1. Divisor igual a cero . . . 15

4. Análisis de resultados **15**

5. Investigación complementaria **18**
5.1. Directivas EQU y DB . . . 18
5.2. Multiplicación de dos números de dos bytes . . . 18
5.3. División de dos números de dos bytes . . . 18
5.4. Suma de dos números de 4 bytes . . . 19
5.4.1. Con registros de 16 bits . . . 19
5.4.2. Con registros de 8 bits . . . 20

6. Conclusiones **22**

7. Bibliografía **22**

1. Objetivos

Objetivo general. Analizar la forma en que se ejecutan algunas instrucciones aritméticas y de manejo de la pila.

Objetivos específicos.

- Observar el uso de algunas instrucciones aritméticas y las banderas que son afectadas.
- Identificar las banderas que afecta cada instrucción observada.
- Utilizar correctamente los registros del microprocesador al realizar operaciones aritméticas, tanto de 8 bits como de 16 bits.
- Analizar detenidamente el proceso de cada operación.

2. Materiales y equipos

- Guía de laboratorio.
- Computadora PC con el programa EMU8086.

3. Actividades

3.1. Preparación del entorno

- Se ejecutó el emulador EMU8086 haciendo doble clic en su icono del escritorio y se creó un nuevo documento tipo COM.
- Cada programa se escribió con su encabezado `#make_COM#` y `org 100h`, necesario para generar un archivo `.COM`.
- Para observar el efecto de cada instrucción se usó la ejecución paso a paso (*single step*), junto con las ventanas **flags** y **memory** del emulador.

Las banderas que se monitorearon se resumen en la Tabla 1 y las instrucciones aritméticas estudiadas en la Tabla 2.

Cuadro 1: Banderas del registro de estados utilizadas en la práctica.

Bandera	**Nombre**	**Se activa (=1) cuando...**
O	Desbordamiento (*overflow*)	el resultado, interpretado con signo, no cabe en el tamaño del operando.
S	Signo	el bit más significativo del resultado es 1 (bit 7 en operaciones de 8 bits, bit 15 en las de 16 bits).
Z	Cero	el resultado de la operación es 0.
A	Acarreo auxiliar	hay acarreo (suma) o préstamo (resta) entre el bit 3 y el bit 4.
P	Paridad	el byte bajo del resultado tiene un número par de unos.
C	Acarreo (*carry*)	hay acarreo fuera del bit más significativo (suma) o préstamo (resta), interpretando los datos sin signo.

Cuadro 2: Instrucciones aritméticas estudiadas.

Instrucción	Operación	Banderas y observaciones
`ADD`	destino ← destino + fuente	Modifica O, S, Z, A, P y C. Ignora el acarreo previo.
`ADC`	destino ← destino + fuente + C	Modifica O, S, Z, A, P y C. Suma el acarreo de la operación anterior.
`SUB`	destino ← destino − fuente	Modifica O, S, Z, A, P y C. C = 1 indica préstamo.
`SBB`	destino ← destino − fuente − C	Modifica O, S, Z, A, P y C. Resta el préstamo de la operación anterior.
`MUL`	AX ← AL × fuente (8 bits, sin signo)	Solo C y O (= 1 si AH ≠ 0). S, Z, A y P quedan indefinidas.
`IMUL`	AX ← AL × fuente (8 bits, con signo)	Solo C y O (= 1 si el producto no cabe en AL con signo). Las demás quedan indefinidas.
`DIV`	AL ← AX ÷ fuente, AH ← residuo (sin signo)	Todas las banderas quedan indefinidas. Error si el divisor es 0 o el cociente supera FFh.
`IDIV`	AL ← AX ÷ fuente, AH ← residuo (con signo)	Todas las banderas quedan indefinidas. Error si el divisor es 0 o el cociente sale de −128 a 127.

3.2. Primer programa: suma con ADD y ADC

Objetivo. Observar la suma de dos números de 1 byte y de dos números de 2 bytes, cómo se actualizan las banderas y qué diferencia hay entre `ADD` y `ADC`. Los datos se declaran con la directiva `EQU` (Tabla 3).

Cuadro 3: Datos declarados en el primer programa.

Constante	Decimal	Hexadecimal	Tamaño
`byte1`	125	7Dh	8 bits
`byte2`	150	96h	8 bits
`word1`	8355	20A3h	16 bits
`word2`	22840	5938h	16 bits

Funcionamiento.

- `AL = byte1 + byte2` (8 bits): 125 + 150 = 275, que no cabe en un byte. AL queda en 13h y el noveno bit pasa a la bandera C.
- `AX = word1 + word2` (16 bits): 20A3h + 5938h = 79DBh.
- Los resultados se copian a memoria: AL en la dirección 011Eh y AX en 011Fh y 0120h (*little endian*: DBh en 011Fh y 79h en 0120h).

```
1  #make_COM#
2  org 100h
3  byte1 equ 125
4  byte2 equ 150
5  word1 equ 20A3h
6  word2 equ 5938h
7
8      mov al, byte1        ; suma de 2 numeros de 1 byte
9      add al, byte2        ; AL = 7Dh + 96h = 13h (con acarreo)
10     mov [011Eh], al      ; M[011Eh] = AL
```

```
11
12      mov ax, word1        ; suma de 2 numeros de 2 bytes
13      add ax, word2        ; AX = 20A3h + 5938h = 79DBh
14      mov [011Fh], ax      ; M[011Fh] = DBh, M[0120h] = 79h
15      ret
16  END
```

Insertar captura: código fuente del primer programa
`img/p1-codigo.png`

Figura 1: Código del primer programa en el editor de EMU8086.

Procedimiento.

- Se digitó y emuló el programa. Antes de ejecutarlo se anotó el contenido de las direcciones 011Eh, 011Fh y 0120h y el estado de las banderas (Figura 2).
- Se ejecutó paso a paso, anotando en cada momento la memoria y las banderas (Tabla 4).

Insertar captura: emulador antes de ejecutar el programa, con la ventana memory y las banderas
`img/p1-inicio.png`

Figura 2: Estado inicial de la memoria (011Eh a 0120h) y de las banderas.

Cuadro 4: Tabla 1: memoria y banderas durante la ejecución del primer programa (con `ADD`).

Momento	**011Eh**	**011Fh**	**0120h**	**O**	**S**	**Z**	**A**	**P**	**C**
Inicio	00h	00h	00h	0	0	0	0	0	0
Después de la 1.ª suma	00h	00h	00h	0	0	0	1	0	1
Antes de la 2.ª suma	13h	00h	00h	0	0	0	1	0	1
Después de la 2.ª suma	13h	00h	00h	0	0	0	0	1	0
Después del último `mov`	13h	DBh	79h	0	0	0	0	1	0

«Antes de la 2.ª suma» es después de ejecutar `mov [011Eh], al` y `mov ax, word1`; `MOV` no modifica banderas.

Respuestas a las preguntas de la guía.

Insertar captura: emulador después de add al, byte2
`img/p1-suma1.png`

Figura 3: Registros, memoria y banderas después de la primera suma.

Insertar captura: emulador después de add ax, word2 y del último mov
`img/p1-suma2.png`

Figura 4: Registros, memoria y banderas después de la segunda suma y del último `mov`.

- *¿Con qué datos se realizan las operaciones y cuáles son los resultados?* Con 7Dh + 96h (125 + 150) en 8 bits, cuyo resultado es 13h con acarreo, y con 20A3h + 5938h en 16 bits, cuyo resultado es 79DBh.
- *¿Dónde se almacena el resultado al final?* En memoria: el byte 13h en 011Eh y la palabra 79DBh en 011Fh (DBh) y 0120h (79h). AX conserva además 79DBh.
- *¿Concuerda el resultado con lo esperado?* Sí. El resultado de 8 bits es 275 − 256 = 19 = 13h, y el 256 que sobra es el acarreo que se ve en C.

3.2.1. Acarreo invertido con `ADD`

Se volvió a cargar el programa y se invirtió manualmente la bandera C (de 0 a 1) antes de ejecutar (equivale a insertar `stc` al inicio). Se ejecutó hasta la primera suma y luego hasta la segunda.

Cuadro 5: Primer programa con `ADD` y C = 1 al inicio.

Instrucción	Operandos	Resultado	O	S	Z	A	P	C
`add al, byte2`	7Dh, 96h	13h	0	0	0	1	0	1
`add ax, word2`	20A3h, 5938h	79DBh	0	0	0	0	1	0

Observaciones. El resultado de ambas sumas **no cambió**: `ADD` no usa el valor previo de C, solo lo actualiza con el acarreo que produce la propia operación. En la segunda suma C pasó a 0 aunque antes valía 1.

3.2.2. Variante con `ADC`

Se sustituyeron las dos instrucciones `ADD` por `ADC` y se repitió el procedimiento desde la inversión del acarreo:

Insertar captura: emulador con C=1 antes de las sumas ADD
`img/p1-add-cf.png`

Figura 5: Resultados de `ADD` con el acarreo activado manualmente al inicio.

```
    mov al, byte1
    adc al, byte2       ; AL = AL + byte2 + C
    mov [011Eh], al

    mov ax, word1
    adc ax, word2       ; AX = AX + word2 + C
    mov [011Fh], ax
    ret
```

Cuadro 6: Primer programa con `ADC` y C = 1 al inicio.

Instrucción	Operandos	Resultado	O	S	Z	A	P	C
`adc al, byte2`	7Dh + 96h + 1	14h	0	0	0	1	1	1
`adc ax, word2`	20A3h + 5938h + 1	79DCh	0	0	0	0	0	0

Insertar captura: emulador con las instrucciones ADC ejecutadas
`img/p1-adc.png`

Figura 6: Resultados del primer programa con `ADC`.

Observaciones. Con `ADC` ambos resultados aumentaron en 1 respecto a `ADD`: 13h pasó a 14h y 79DBh pasó a 79DCh, porque la instrucción suma además el acarreo. Las banderas A y P cambiaron porque dependen del resultado: 14h tiene dos unos (P = 1) y 79DCh tiene cinco (P = 0). Aunque se empiece con C = 0, la segunda suma también da 79DCh, ya que recibe el acarreo C = 1 que dejó la primera.

3.2.3. Datos modificados

Se restauró `ADD` y se cambiaron los datos por: `byte1 equ 9Dh`, `byte2 equ 83h`, `word1 equ 5938` y `word2 equ -5938` (5938 = 1732h y −5938 = E8CEh en complemento a 2).

Cuadro 7: Primer programa con los datos modificados (`ADD`).

Instrucción	Operandos	Resultado	O	S	Z	A	P	C
`add al, byte2`	9Dh, 83h	20h (120h)	1	0	0	1	0	1
`add ax, word2`	1732h, E8CEh	0000h (10000h)	0	0	1	1	1	1

Entre paréntesis, el resultado completo incluyendo el acarreo.

Insertar captura: emulador con los datos 9Dh, 83h, 5938 y -5938
`img/p1-datos-nuevos.png`

Figura 7: Registros, memoria y banderas con los datos modificados.

Observaciones. En memoria quedaron 20h en 011Eh y 0000h en 011Fh y 0120h. Las banderas reaccionaron así:

- *Suma de bytes.* 9Dh + 83h = 120h. C = 1 porque, sin signo, 157 + 131 = 288 supera 255. O = 1 porque, con signo, (−99) + (−125) = −224 no cabe en −128 a 127 (dos negativos dan un resultado positivo). A = 1 porque Dh + 3h = 10h acarrea al nibble alto. S = 0 y Z = 0 porque 20h es positivo y distinto de cero, y P = 0 porque 20h tiene un solo uno.
- *Suma de palabras.* 5938 + (−5938) = 0, es decir, 1732h + E8CEh = 10000h. Z = 1 y P = 1 porque el resultado de 16 bits es 0000h, y S = 0. C = 1 porque, sin signo, la suma rebasa FFFFh. O = 0 porque sumar dos números de signos opuestos nunca desborda. A = 1 porque 2h + Eh = 10h.

3.3. Segundo programa: resta con SUB y SBB

Objetivo. Observar la resta de números de 1 y de 2 bytes, el comportamiento de las banderas C y A según el minuendo sea mayor o menor que el sustraendo, y la diferencia entre `SUB` y `SBB`.

Funcionamiento.

- `CL = byte1 - byte2` (8 bits): 80 − 69 = 11 (50h − 45h = 0Bh).
- `DX = word1 - word2` (16 bits): 26437 − 325 = 26112 (6745h − 0145h = 6600h).
- El programa no copia los resultados a memoria: quedan en los registros CL y DX.

```
#make_COM#
org 100h
byte1 equ 80
byte2 equ 69
word1 equ 26437
word2 equ 325

    mov cl, byte1       ; resta de 2 numeros de 1 byte
    sub cl, byte2       ; CL = 50h - 45h = 0Bh

    mov dx, word1       ; resta de 2 numeros de 2 bytes
    sub dx, word2       ; DX = 6745h - 0145h = 6600h
```

```
    ret
END
```

Insertar captura: código fuente del segundo programa
`img/p2-codigo.png`

Figura 8: Código del segundo programa en el editor de EMU8086.

Cuadro 8: Datos del segundo programa en decimal y en hexadecimal.

Constante	Decimal	Hexadecimal	Papel
byte1	80	50h	minuendo (8 bits)
byte2	69	45h	sustraendo (8 bits)
word1	26437	6745h	minuendo (16 bits)
word2	325	0145h	sustraendo (16 bits)

Procedimiento.

- Se digitó y emuló el programa, anotando los minuendos y sustraendos en hexadecimal (Tabla 8).
- Se ejecutó paso a paso observando los resultados y las banderas (**1.ª prueba**, `SUB`).
- Se invirtió el orden de la operación, de modo que el minuendo pasa a ser sustraendo y viceversa (**2.ª prueba**, `SUB`):

```
    mov cl, byte2        ; ahora byte2 es el minuendo
    sub cl, byte1        ; CL = 45h - 50h = F5h (hay prestamo)
    mov dx, word2
    sub dx, word1        ; DX = 0145h - 6745h = 9A00h
```

- Se restauró el programa original y se sustituyeron las instrucciones `SUB` por `SBB`, repitiendo las dos pruebas (**3.ª y 4.ª prueba**):

```
    mov cl, byte1
    sbb cl, byte2        ; CL = CL - byte2 - C
    mov dx, word1
    sbb dx, word2        ; DX = DX - word2 - C
```

3.3.1. Acarreo activado manualmente

Se repitieron las pruebas activando manualmente la bandera C antes de cada resta (equivale a insertar `stc` antes de cada `sub` o `sbb`).

Observaciones.

Cuadro 9: Tabla 2: resultados de las restas con `SUB` y `SBB` (C = 0 al inicio).

Prueba	Instrucción	Minuendo	Sustraendo	Diferencia	O	S	Z	A	P	C
1.ª `SUB`	`sub cl, byte2`	50h	45h	0Bh	0	0	0	1	0	0
1.ª `SUB`	`sub dx, word2`	6745h	0145h	6600h	0	0	0	0	1	0
2.ª `SUB`	`sub cl, byte1`	45h	50h	F5h	0	1	0	0	1	1
2.ª `SUB`	`sub dx, word1`	0145h	6745h	9A00h	0	1	0	0	1	1
3.ª `SBB`	`sbb cl, byte2`	50h	45h	0Bh	0	0	0	1	0	0
3.ª `SBB`	`sbb dx, word2`	6745h	0145h	6600h	0	0	0	0	1	0
4.ª `SBB`	`sbb cl, byte1`	45h	50h	F5h	0	1	0	0	1	1
4.ª `SBB`	`sbb dx, word1`	0145h	6745h	99FFh	0	1	0	1	1	1

Insertar captura: emulador después de las dos restas SUB en el orden original
`img/p2-sub1.png`

Figura 9: Registros y banderas después de las restas con `SUB` (1.ª prueba).

Insertar captura: emulador después de las restas SUB con minuendo y sustraendo invertidos
`img/p2-sub2.png`

Figura 10: Registros y banderas con `SUB` en orden invertido (2.ª prueba).

Insertar captura: emulador después de las restas SBB en el orden original
`img/p2-sbb3.png`

Figura 11: Registros y banderas con `SBB` en el orden original (3.ª prueba).

Insertar captura: emulador después de las restas SBB con minuendo y sustraendo invertidos
`img/p2-sbb4.png`

Figura 12: Registros y banderas con `SBB` en orden invertido (4.ª prueba).

Cuadro 10: Restas con C = 1 antes de cada operación.

Instrucción	**Minuendo**	**Sustraendo**	**Diferencia**	**O**	**S**	**Z**	**A**	**P**	**C**
`sub cl, byte2`	50h	45h	0Bh	0	0	0	1	0	0
`sub dx, word2`	6745h	0145h	6600h	0	0	0	0	1	0
`sub cl, byte1`	45h	50h	F5h	0	1	0	0	1	1
`sub dx, word1`	0145h	6745h	9A00h	0	1	0	0	1	1
`sbb cl, byte2`	50h	45h	0Ah	0	0	0	1	1	0
`sbb dx, word2`	6745h	0145h	65FFh	0	0	0	1	1	0
`sbb cl, byte1`	45h	50h	F4h	0	1	0	0	0	1
`sbb dx, word1`	0145h	6745h	99FFh	0	1	0	1	1	1

Insertar captura: emulador con C=1 antes de las restas SUB y SBB
`img/p2-acarreo.png`

Figura 13: Resultados de las restas con el acarreo activado manualmente.

- Con `SUB` el acarreo previo **no cambia** ninguna operación (Tabla 10 es igual a la Tabla 9 en las filas `SUB`).
- Con `SBB` sí cambia: cada diferencia disminuye en 1 (0Bh pasa a 0Ah, 6600h pasa a 65FFh, F5h pasa a F4h), porque la instrucción resta además el préstamo.
- Con `SBB` en orden invertido y C = 0 al inicio, la segunda resta ya da 99FFh en lugar de 9A00h: la primera resta dejó C = 1 (préstamo) y la segunda lo recibió.

3.4. Tercer programa: multiplicación con MUL e IMUL

Objetivo. Observar la multiplicación de dos números de 1 byte, en qué registros se guardan los datos y el producto, y la diferencia entre multiplicar sin signo (`MUL`) y con signo (`IMUL`).

Funcionamiento.

- AH = AAh y AL = FFh. El valor de AH se pierde, porque el producto de 16 bits sobrescribe todo AX.
- CL = 06h (multiplicador).
- `mul cl`: AX = AL × CL = FFh × 06h = 05FAh (255 × 6 = 1530).

```
#make_COM#
org 100h
INICIO:
    mov ah, 0AAh        ; multiplica 2 numeros de 1 byte
    mov al, 0FFh        ; multiplicando
    mov cl, 06          ; multiplicador (dato)
    mul cl              ; AX = AL x CL
    ret
END
```

Insertar captura: código fuente del tercer programa
`img/p3-codigo.png`

Figura 14: Código del tercer programa en el editor de EMU8086.

Respuestas a las preguntas de la guía.

- *¿Qué registros están involucrados?* AL (multiplicando, implícito), CL (multiplicador) y AX (producto). AH solo se carga con AAh, pero no participa.
- *¿Qué datos se operan?* FFh y 06h.
- *¿Dónde se almacena el resultado?* En AX (AH = parte alta, AL = parte baja).

Procedimiento. Se ejecutó paso a paso con el dato 06 y con `MUL` (Figura 15); luego con CL = −06 (Figura 16); después se cambió `mul cl` por `imul cl` y se probó con 06 (Figura 17) y con −06 (Figura 18).

Observaciones. Los datos son los mismos bits en las cuatro pruebas (FFh y 06h o FAh), pero `MUL` los interpreta como números sin signo e `IMUL` como números con signo, y por eso los productos son distintos. Con `MUL` la parte alta del producto (AH ≠ 0) activa C y O. Con `IMUL` el producto con signo cabe en AL, así que C y O quedan en 0.

Cuadro 11: Tabla 3: multiplicación con `MUL` e `IMUL`.

Instrucción	**Dato**	**Multiplicando (AL)**	**Multiplicador (CL)**	**Producto (AX)**	**C/O**
`MUL`	06	FFh = 255	06h = 6	05FAh = 1530	1/1
`MUL`	−06	FFh = 255	FAh = 250	F906h = 63750	1/1
`IMUL`	06	FFh = −1	06h = 6	FFFAh = −6	0/0
`IMUL`	−06	FFh = −1	FAh = −6	0006h = 6	0/0

La columna C/O no la pide la guía, pero ayuda en el análisis. Las demás banderas quedan indefinidas.

Insertar captura: emulador después de mul cl con 06
`img/p3-mul.png`

Figura 15: Resultado de `mul cl` con CL = 06.

Insertar captura: emulador después de mul cl con -06
`img/p3-mul-neg.png`

Figura 16: Resultado de `mul cl` con CL = −06 (FAh).

Insertar captura: emulador después de imul cl con 06
`img/p3-imul.png`

Figura 17: Resultado de `imul cl` con CL = 06.

Insertar captura: emulador después de imul cl con -06

`img/p3-imul-neg.png`

Figura 18: Resultado de `imul cl` con CL = −06 (FAh).

3.5. Cuarto programa: división con DIV e IDIV

Objetivo. Observar la división de un número de 16 bits entre uno de 8 bits, en qué registros quedan el cociente y el residuo, y la diferencia entre dividir sin signo (`DIV`) y con signo (`IDIV`).

Funcionamiento.

- AX = 123 = 007Bh (dividendo de 16 bits).
- CL = 05h (divisor, el dato).
- `div cl`: AL = cociente = 123 ÷ 5 = 24 (18h) y AH = residuo = 3 (03h). AX queda en 0318h.

```
#make_COM#
org 100h
INICIO:
    mov ax, 123          ; dividendo de 16 bits
    mov cl, 05           ; divisor (dato)
    div cl               ; AL = AX / CL (cociente), AH = AX mod CL (residuo)
    ret
END
```

Insertar captura: código fuente del cuarto programa

`img/p4-codigo.png`

Figura 19: Código del cuarto programa en el editor de EMU8086.

Respuestas a las preguntas de la guía.

- *¿Qué registros están involucrados?* AX (dividendo), CL (divisor), AL (cociente) y AH (residuo).
- *¿Qué datos se operan?* 123 (007Bh) y 05h.
- *¿Dónde se almacena el resultado?* En AX: el cociente en AL y el residuo en AH.

Procedimiento. Se siguió el mismo procedimiento del tercer programa: `DIV` con 05 y con −05 (Figuras 20 y 21); luego se cambió a `IDIV` y se probó con 05 y con −05 (Figuras 22 y 23).

Cuadro 12: Tabla 4: división con `DIV` e `IDIV`.

Instrucción	Dato	Dividendo (AX)	Divisor (CL)	Cociente (AL)	Residuo (AH)
`DIV`	05	007Bh = 123	05h = 5	18h = 24	03h = 3
`DIV`	−05	007Bh = 123	FBh = 251	00h = 0	7Bh = 123
`IDIV`	05	007Bh = +123	05h = +5	18h = +24	03h = +3
`IDIV`	−05	007Bh = +123	FBh = −5	E8h = −24	03h = +3

Las banderas quedan indefinidas después de `DIV` e `IDIV`.

Insertar captura: emulador después de div cl con 05
`img/p4-div.png`

Figura 20: Resultado de `div cl` con CL = 05.

Insertar captura: emulador después de div cl con -05
`img/p4-div-neg.png`

Figura 21: Resultado de `div cl` con CL = −05 (FBh).

3.5.1. Divisor igual a cero

Se cambió el dato por 00h (`mov cl, 00h`) y se ejecutó el programa.

Observaciones. Al ejecutar `div cl` con CL = 00h el programa **no termina normalmente**: el emulador muestra un error de división (*divide error*) y se detiene. La división entre cero no está definida, y el procesador responde generando la interrupción de tipo 0 en lugar de entregar un cociente.

4. Análisis de resultados

a) **Primer programa.**

- **Banderas con los datos originales.**

Insertar captura: emulador después de idiv cl con 05
`img/p4-idiv.png`

Figura 22: Resultado de `idiv cl` con CL = 05.

Insertar captura: emulador después de idiv cl con -05
`img/p4-idiv-neg.png`

Figura 23: Resultado de `idiv cl` con CL = −05 (FBh).

Insertar captura: mensaje de error (divide error) del emulador al dividir entre cero
`img/p4-error.png`

Figura 24: Error del emulador al dividir entre 00h.

  - *Suma de bytes* (7Dh + 96h = 113h, AL = 13h). C = 1 porque 125 + 150 = 275 supera 255. A = 1 porque Dh + 6h = 13h acarrea del bit 3 al 4. O = 0 porque 7Dh es positivo (+125) y 96h es negativo (−106), y sumar números de signo distinto no desborda (125 − 106 = 19). S = 0 (el bit 7 de 13h es 0), Z = 0 y P = 0 (13h = 0001 0011b tiene tres unos).
  - *Suma de palabras* (20A3h + 5938h = 79DBh). C = 0 porque no se pasa de FFFFh. A = 0 porque 3h + 8h = Bh no acarrea. O = 0 porque dos positivos dan 79DBh, que sigue siendo positivo (menor que 8000h). S = 0, Z = 0 y P = 1 (el byte bajo DBh = 1101 1011b tiene seis unos).

- **Banderas con los datos del paso 12.** 9Dh + 83h = 120h, con C = 1 (157 + 131 = 288 ¿255), O = 1 (−99 − 125 = −224 no cabe en un byte con signo) y A = 1. En 16 bits, 1732h + E8CEh = 10000h, con Z = 1 y P = 1 (el resultado de 16 bits es 0000h), C = 1 y O = 0 (sumar un número y su opuesto no desborda).
- **¿Por qué cambiaron los resultados con `ADC` en lugar de `ADD`?** `ADD` calcula destino + fuente y `ADC` calcula destino + fuente + C. Con C = 1 al inicio: 7Dh + 96h + 1 = 114h, es decir AL = 14h (en lugar de 13h). Como la primera suma vuelve a dejar C = 1, la segunda es 20A3h + 5938h + 1 = 79DCh (en lugar de 79DBh). Por eso `ADC` se usa para encadenar sumas de varios bytes: arrastra el acarreo de la parte baja a la parte alta.

b) **Segundo programa.**

- **C y A según el minuendo sea mayor o menor.** En la resta, C = 1 indica que hubo *préstamo*: el minuendo es menor que el sustraendo y se tuvo que «pedir prestado» un 1 al bit siguiente al más significativo. Con 50h − 45h (minuendo mayor) no hay préstamo y C = 0. Con 45h − 50h (minuendo menor) se calcula (100h + 45h) − 50h = F5h y C = 1. De forma análoga A = 1 cuando el nibble bajo del minuendo es menor que el del sustraendo: en 50h − 45h, 0h ¡5h, se pide prestado al nibble alto y A = 1; en 45h − 50h, 5h ≥ 0h y A = 0 (el préstamo ocurre en el nibble alto, 4h ¡5h, y por eso C = 1). Las restas de 16 bits siguen la misma lógica: 6745h − 0145h no pide prestado (C = 0), mientras que 0145h − 6745h = 10145h − 6745h = 9A00h sí lo pide (C = 1).
- **¿Por qué cambiaron los resultados con `SBB` en lugar de `SUB`?** `SBB` calcula destino−fuente−C, es decir, resta además el préstamo anterior. Con C = 0 la primera resta es igual que con `SUB` (0Bh y 6600h). En el orden invertido, la resta de bytes deja C = 1 y la siguiente es 0145h − 6745h − 1 = 99FFh (en lugar de 9A00h). Si C se activa a mano antes de cada resta, todos los resultados disminuyen en 1: 50h − 45h − 1 = 0Ah, 6745h − 0145h − 1 = 65FFh y 45h − 50h − 1 = F4h. Así, `SBB` permite encadenar restas de varios bytes.

c) **Tercer programa.**

- **¿Cómo se especifica que se trabaja con números de un byte?** Por el tamaño del operando que acompaña a la instrucción: `mul cl` tiene un operando de 8 bits (CL) y, por tanto, el otro factor está implícito en AL. Si el operando fuera de 16 bits (por ejemplo `mul cx`), se multiplicaría AX × CX.
- **Registros que se usan.** El multiplicando va en AL, el multiplicador en un registro o posición de memoria de 8 bits (CL) y el producto de 16 bits queda en AX. (En multiplicaciones de 16 bits el producto queda en el par DX:AX.)
- **¿Por qué cambiaron los resultados con `IMUL` en lugar de `MUL`?** Los bits son los mismos, pero la interpretación es distinta. Con 06: `MUL` calcula 255 × 6 = 1530 = 05FAh e `IMUL` calcula (−1) × 6 = −6 = FFFAh. Con −06 (FAh): `MUL` calcula 255 × 250 = 63750 = F906h e `IMUL` calcula (−1) × (−6) = 6 = 0006h. Además, con `MUL` se activan C y O porque AH ≠ 0, mientras que con `IMUL` se quedan en 0 porque el producto con signo cabe en AL.

d) **Cuarto programa.**

- **¿Cómo se especifica que se trabaja con números de un byte?** También por el tamaño del divisor: `div cl` tiene un divisor de 8 bits, por lo que el dividendo es AX. Con un divisor de 16 bits (por ejemplo `div cx`) el dividendo sería DX:AX.

- **Registros que se usan.** Dividendo de 16 bits en AX, divisor de 8 bits en CL, cociente en AL y residuo en AH. (Con divisor de 16 bits: dividendo en DX:AX, cociente en AX y residuo en DX.)
- **¿Por qué cambiaron los resultados con `IDIV` en lugar de `DIV`?** Con 05 ambas instrucciones dan lo mismo, porque todos los datos son positivos: $123 \div 5 = 24$ con residuo 3 (0318h). Con -05 cambian. `DIV` ve CL = FBh como 251 y calcula $123 \div 251 = 0$ con residuo 123 (AX = 7B00h). `IDIV` ve CL como -5 y calcula $123 \div (-5) = -24$ con residuo $+3$ (AL = E8h, AH = 03h, AX = 03E8h); se comprueba con $(-24)(-5) + 3 = 123$. El residuo de `IDIV` lleva el signo del dividendo.
- **¿Por qué la última modificación generó un error?** Porque con CL = 00h se intenta dividir entre cero, y esa operación no está definida. El procesador genera la excepción de división (interrupción tipo 0) y el emulador detiene el programa. El mismo error ocurre si el cociente no cabe en AL (más de FFh con `DIV`, o fuera de -128 a 127 con `IDIV`).

5. Investigación complementaria

5.1. Directivas EQU y DB

Cuadro 13: Comparación entre las directivas `EQU` y `DB`.

Característica	`EQU`	`DB`
Qué hace	Asigna un nombre a un valor constante.	Reserva y asigna valor inicial a bytes de memoria.
Memoria	No reserva memoria: el ensamblador reemplaza el nombre por el valor.	Reserva 1 byte por cada valor.
Uso en la instrucción	Operando inmediato (`mov al, byte1` se ensambla como `mov al, 7Dh`).	Operando de memoria (direccionamiento directo): `mov al, dato`.
Modificable al ejecutar	No, es una constante.	Sí, se puede leer y escribir (`mov dato, al`).
Dirección	No tiene.	La etiqueta equivale al desplazamiento donde queda el dato.

5.2. Multiplicación de dos números de dos bytes

Se usa la forma de 16 bits de `MUL` (o `IMUL` si hay signo): el multiplicando va en AX, el multiplicador en un registro de 16 bits y el producto de 32 bits queda en el par DX:AX (DX = parte alta, AX = parte baja).

```
1    mov ax, 1234h        ; multiplicando
2    mov bx, 0100h        ; multiplicador
3    mul bx               ; DX:AX = 1234h x 0100h = 0012 3400h
```

Como DX $\neq 0$, las banderas C y O quedan en 1. También puede hacerse con registros de 8 bits mediante productos parciales: $(a_H 2^8 + a_L)(b_H 2^8 + b_L) = a_H b_H 2^{16} + (a_H b_L + a_L b_H)2^8 + a_L b_L$, con cuatro `MUL` de 8 bits y sumas encadenadas con `ADC`.

5.3. División de dos números de dos bytes

Se usa la forma de 16 bits de `DIV` (o `IDIV`): el dividendo de 32 bits va en DX:AX, el divisor en un registro de 16 bits, el cociente queda en AX y el residuo en DX. Si el dividendo solo tiene 16 bits, hay que completar DX: con 0 para `DIV` (`xor dx, dx`) o con la extensión de signo para `IDIV` (`cwd`).

```
1    mov ax, 5000         ; dividendo (16 bits)
2    xor dx, dx           ; DX = 0, completa el dividendo a 32 bits
```

```
    mov bx, 7              ; divisor
    div bx                 ; AX = 02CAh (714), DX = 0002h (residuo)
```

Se verifica con $714 \times 7 + 2 = 5000$. Igual que en 8 bits, hay error de división si el divisor es 0 o si el cociente no cabe en AX.

5.4. Suma de dos números de 4 bytes

Se suman A = 0012FFFFh y B = 00340001h. Como ningún registro del 8086 guarda 32 bits, la suma se hace por partes y el acarreo de cada parte se arrastra a la siguiente con `ADC`: la parte menos significativa se suma con `ADD` y las demás con `ADC`. En ambos casos el resultado esperado es R = 00470000h.

5.4.1. Con registros de 16 bits

Se suman dos palabras: primero la baja con `ADD` y luego la alta con `ADC`. Entre ambas solo hay instrucciones `MOV`, que no alteran C.

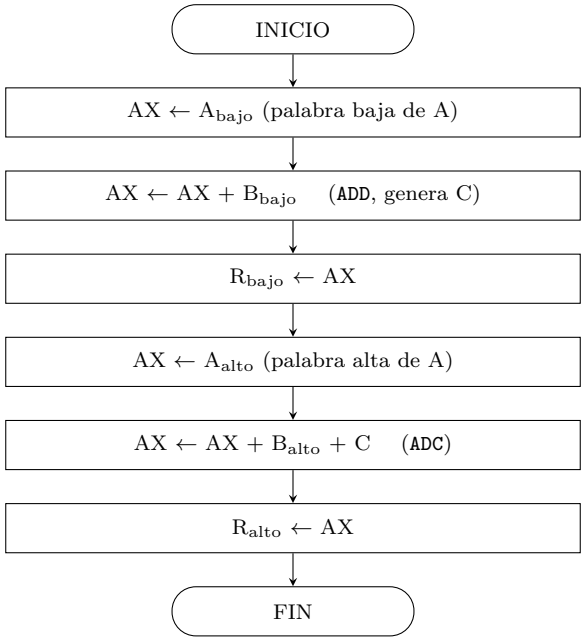


Figura 25: Diagrama de flujo de la suma de dos números de 4 bytes con registros de 16 bits.

```
#make_COM#
org 100h
INICIO:
    mov ax, a_baja
    add ax, b_baja      ; FFFFh + 0001h = 0000h, C = 1
    mov r_baja, ax
    mov ax, a_alta      ; MOV no altera C
    adc ax, b_alta      ; 0012h + 0034h + 1 = 0047h
    mov r_alta, ax
    ret

a_baja dw 0FFFFh
a_alta dw 0012h
b_baja dw 0001h
b_alta dw 0034h
r_baja dw 0
```

```
r_alta dw 0
END
```

Insertar captura: emulador con el resultado 0047h 0000h de la suma de 32 bits
`img/inv-16bits.png`

Figura 26: Ejecución de la suma de 4 bytes con registros de 16 bits.

5.4.2. Con registros de 8 bits

Se suman cuatro bytes, del menos al más significativo: el primero con `ADD` y los otros tres con `ADC`.

```
#make_COM#
org 100h
INICIO:
    mov al, numA_0
    add al, numB_0      ; FFh + 01h = 00h, C = 1
    mov res_0, al
    mov al, numA_1
    adc al, numB_1      ; FFh + 00h + 1 = 00h, C = 1
    mov res_1, al
    mov al, numA_2
    adc al, numB_2      ; 12h + 34h + 1 = 47h, C = 0
    mov res_2, al
    mov al, numA_3
    adc al, numB_3      ; 00h + 00h + 0 = 00h
    mov res_3, al
    ret

numA_0 db 0FFh
numA_1 db 0FFh
numA_2 db 12h
numA_3 db 00h
numB_0 db 01h
numB_1 db 00h
numB_2 db 34h
numB_3 db 00h
res_0  db 0
res_1  db 0
res_2  db 0
res_3  db 0
END
```

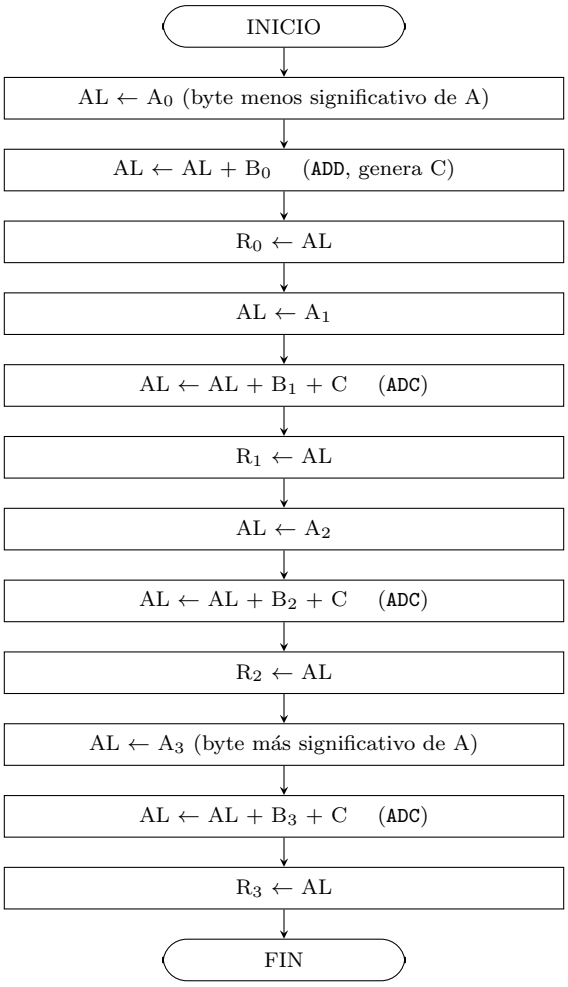


Figura 27: Diagrama de flujo de la suma de dos números de 4 bytes con registros de 8 bits.

Insertar captura: emulador con los bytes 00h 00h 47h 00h del resultado
`img/inv-8bits.png`

Figura 28: Ejecución de la suma de 4 bytes con registros de 8 bits.

6. Conclusiones

- `ADD` y `SUB` solo usan los operandos, mientras que `ADC` y `SBB` incluyen además el acarreo o préstamo previo. Por eso estas últimas permiten operar números más grandes que un registro, encadenando partes de menor a mayor peso.
- La bandera C señala el desbordamiento de números *sin signo* (acarreo en la suma, préstamo en la resta) y la bandera O el de números *con signo*. Un mismo resultado puede activar una, la otra, ambas o ninguna: 9Dh + 83h activó las dos, mientras que 7Dh + 96h activó solo C.
- En la resta, C = 1 indica préstamo (minuendo menor que sustraendo) y A = 1 indica préstamo entre nibbles; el resultado queda en complemento a 2 (45h − 50h = F5h).
- El tamaño de una multiplicación o división lo fija el operando: con un operando de 8 bits el otro dato está implícito en AL (`MUL`) o en AX (`DIV`); con uno de 16 bits se usa AX o DX:AX. El producto de `MUL` ocupa el doble de bits de los factores.
- `MUL` y `DIV` interpretan los datos sin signo, e `IMUL` e `IDIV` con signo. Con los mismos bits (FFh y FAh) los resultados difieren: 1530 frente a −6 en la multiplicación, y 0 frente a −24 en la división. En `IDIV` el residuo lleva el signo del dividendo.
- Dividir entre cero, o con un cociente que no cabe en el registro de destino, genera un error de ejecución (interrupción tipo 0).
- `EQU` define constantes que el ensamblador sustituye en el código, mientras que `DB` reserva memoria con una dirección y un valor que se puede modificar.
- Las instrucciones aritméticas modifican las banderas (O, S, Z, A, P, C) según el resultado, salvo `MUL`, `IMUL`, `DIV` e `IDIV`, que dejan indefinidas varias de ellas.

7. Bibliografía

- Guía de laboratorio de Arquitectura de Computadoras, Práctica 5: Programación para el microprocesador INTEL, instrucciones aritméticas. Escuela Profesional de Ingeniería de Sistemas.
- Brey, B. B. *Los microprocesadores Intel*. Pearson Educación.
- Intel Corporation. *The 8086 Family User's Manual*.
- Documentación (ayuda) del emulador EMU8086.